

Comprehensive Technical Report

UI/UX features · workflow & system architecture · AI & MCP integration
security measures · testing & QA · deployment pipeline

System	JSD Emergency Services — ambulance/fire dispatch platform for Tata Steel, Jamshedpur
Frontend	React 18 + Vite + Tailwind CSS, Recharts, Leaflet, Zustand
Backend	.NET 8 AWS Lambda (TransportApi, VoiceAgent), DynamoDB, API Gateway HTTP API
Hosting	S3 + CloudFront (SPA), OIDC-based GitHub Actions / CodeBuild deploy
AI runtime	Amazon Bedrock — Nova Lite (<code>eu.amazon.nova-lite-v1:0</code>)
Report basis	Direct source inspection — every material claim is cited to a file/line

Contents

1. Executive summary
2. System architecture overview
3. UI/UX — pages & features
4. UI/UX — interaction mechanisms
5. UI/UX — component & state architecture
6. Backend — API endpoint inventory
7. Backend — dispatch & assignment algorithm
8. Backend — DynamoDB schema
9. Live routing & traffic simulation
10. AI features: VoiceAgent & OKF
11. AI features: MCP integration
12. AI features: the "policy engine" & RAG status
13. AI features: statistical analytics (coverage-gap, AI Insights)
14. Security architecture
15. Testing & QA
16. Deployment pipeline
17. Known gaps & technical debt
18. Summary matrix

1. Executive summary

JSD Emergency Services is a full-stack ambulance/fire dispatch platform: a React SPA (three role-based surfaces — Control Room console, requester Portal, public tracking link) backed by two .NET 8 Lambda functions (TransportApi, VoiceAgent) and six DynamoDB tables, fronted by API Gateway and served via S3/CloudFront. It layers a genuine but narrow use of generative AI (Bedrock Nova Lite) onto a deterministic, auditable dispatch core — the LLM extracts structured intent from natural language, but every value it proposes is checked against closed enums or real reference data before anything is dispatched or reconfigured.

This report is organized to answer four questions precisely: what does the product actually do (§3-§5), how is it built (§6-§9), what AI is really in it and what isn't (§10-§13), and how well is it secured and tested (§14-§15). Every AI-adjacent claim distinguishes three categories that are easy to conflate: **generative AI** (an LLM performs inference — 2 places), **statistical/deterministic analytics** (aggregation, Little's Law, centroid geometry — no model inference, 2 endpoints), and **not implemented** (named suggestively or documented as a proposal, but no working code exists — most notably, there is no retrieval-augmented generation anywhere in this codebase).

2. System architecture overview

Layer	Components
Frontend (SPA)	React 18 + Vite, Tailwind CSS, Recharts (charts), Leaflet + CARTO Positron tiles (maps), Zustand (state) — 3 surfaces: admin Console, requester Portal, public Track page
Backend (compute)	<code>lambda/TransportApi</code> (.NET 8, ~1,700 lines, the dispatch/data API), <code>lambda/VoiceAgent</code> (.NET 8, conversational intake), <code>infra/policy-agent/policy-sync.mjs</code> (Node.js, PDF→config extraction, invoked by TransportApi)
Data	6 DynamoDB tables: <code>TransportRequests</code> , <code>Fleet</code> , <code>ReferenceData</code> , <code>ShuttleCards</code> , <code>jamshedpur-users</code> , <code>TransportRequestsHistorySynthetic</code> (isolated, seeded demo data)
Networking	API Gateway HTTP API → both Lambdas; S3 + CloudFront for the SPA and a policy-PDF bucket
Auth	Amazon Cognito (shared with the parent Jamshedpur platform) — JWKS-verified JWTs; separate scoped API keys for server-to-server callers (hospital, fuel, HR, MCP, etc.)
AI	Amazon Bedrock, Nova Lite model, used in exactly two Lambdas (VoiceAgent slot extraction; policy-agent PDF extraction)
Observability	CloudWatch Logs/Metrics + X-Ray, surfaced via an admin Infra Health page and a local MCP dev tool
External routing	Public OSRM demo server (<code>router.project-osrm.org</code>) for real road-network distance/duration, both server-side (dispatch ETA) and client-side (live map animation)

3. UI/UX — pages & features

Route table and nav sections: `src/App.jsx:23-40`, 326-336.

Page	Route	File	What it does
Dashboard	<code>/dashboard</code>	<code>features/dashboard/DashboardPage.jsx</code>	5 animated KPI cards, 6 chart cards (donut/bar/line), live "Active responses" table, and a Coverage Gap Analysis card fed by seeded historical data. Falls back to a Power BI embed if configured.
Emergencies	<code>/emergency</code>	<code>features/emergency/EmergencyPage.jsx</code>	Full-screen Leaflet map with floating header, traffic-mode selector, alerts panel, filterable card list, and a New Emergency drawer (medical/fire/blood, mass-casualty sizing).
Dispatch Board	<code>/requests</code>	<code>features/requests/RequestsPage.jsx</code>	Dense table of all emergencies: KPI-tile filters, search (<code>/</code> shortcut), bulk select + bulk cancel/clear, manual-override modal, pagination (10/page).
Live Map	<code>/map</code>	<code>features/map/LiveMapPage.jsx</code>	Full-screen fleet map: zone polygons, hospital/fire-station markers, live routes, collapsible "Zone Fleet Pools" panel, vehicle detail drawer.
Fleet & Crews	<code>/fleet</code>	<code>features/fleet/FleetPage.jsx</code>	Tabbed Vehicles/Crews/Service Zones view; map + floating vehicle list, service-due detection (10,000 km interval), pagination (8/page).
Power BI	<code>/powerbi</code>	<code>features/dashboard/PowerBIPage.jsx</code>	Dedicated Power BI analytics embed page.
AI Insights	<code>/insights</code>	<code>features/insights/InsightsPage.jsx</code>	Prescriptive recommendation cards (placement, staffing, seasonal alerts) — see §13.
Infra Health	<code>/admin/infra</code>	<code>features/admin/InfraHealthPage.jsx</code>	CloudWatch-backed ops dashboard: time-range selector, 60s auto-refresh, threshold-colored charts, recent error log.
Track (public)	<code>/track/:id</code>	<code>features/track/TrackPage.jsx</code>	Unauthenticated shareable link (bypasses auth entirely, <code>App.jsx:21,74</code>) — live map, OSRM-interpolated vehicle position, ETA countdown.
User Portal	role <code>user</code>	<code>portal/UserPortal.jsx</code>	Requester self-service UI: booking form, live map, "Call for help" voice-agent trigger, request-status tracker (Requested → Unit assigned → En route → Arrived).

4. UI/UX — interaction mechanisms

Mechanism	File	Behavior
Command palette	components/common/CommandPalette.jsx	Ctrl/Cmd+K; static admin nav commands + fuzzy search over live emergencies by id/case type/severity/state; arrow-key nav, <code>role="dialog"</code> <code>aria-modal</code> .
Session idle guard	hooks/useSessionGuard.js	Tracks activity across mouse/keyboard/touch/scroll; expires locally after 25 min idle, or after 60s idle once the SSO token has expired — no forced cross-app redirect.
SWR caching hook	hooks/useCachedApi.js	Instant render from localStorage, background refetch, silent-fail if a cached copy exists. Backs the Coverage Gaps card and AI Insights page.
Bulk actions	features/requests/RequestsPage.jsx:102-118	Checkbox multi-select, bulk cancel/clear, "Clear completed"/"No-facility" bulk buttons; row order freezes while a row menu is open so the 5s poll can't shift it.
Live route rendering	features/emergency/EmergencyPage.jsx:255-332	<code>EmergencyMap</code> wrapped in <code>React.memo</code> ; route geometry (<code>EmergencyRoute</code>) and the moving icon (<code>VehicleMarker</code>) are split so the 1s position tick only re-renders the small marker, not the full polyline — the fix for a previously-reported animation stutter.
Boot / loading states	App.jsx:106-139	Branded "dispatch radar sweep" boot screen; distinct connection-error screen with retry; Portal has its own skeleton-pulse loading state.
Accessibility	throughout	<code>aria-label</code> / <code>aria-modal</code> / <code>role="dialog"</code> on modals; <code>role="tablist"/"tab"</code> on filter groups; <code>aria-hidden</code> on decorative icons; <code>prefers-reduced-motion</code> respected in the KPI count-up animation.
Theming	tailwind.config.js	Brand teal/green (<code>#07514D</code> / <code>#0B6A64</code>), lime CTA (<code>#D6DF27</code>), cream page background (<code>#F7F4EF</code>); CARTO Positron map tiles used consistently across all 5 map surfaces.
Charting	Recharts, throughout	Donut, 5+ bar-chart variants, dual-line time series, stacked bars, inline SVG sparklines. AI Insights deliberately uses zero charts — recommendation cards instead (see §13).

5. UI/UX — component & state architecture

5.1 Presentational patterns

`NeoCard` / `NeoKpi` (titled card + animated-KPI wrappers, duplicated locally in `DashboardPage.jsx:375-400` and `InsightsPage.jsx:140-147`), a single shared `Icon.jsx` dictionary of ~25 real-JSX SVG paths (no `dangerouslySetInnerHTML`), and generic `Donut` / `Bars` / `Empty` chart wrappers reused across every chart card in `DashboardPage.jsx:401-461`. `useCountUp` (inline, `DashboardPage.jsx:340-359`) eases KPI values in via `requestAnimationFrame` and cubic ease-out, runs once per mount, and respects `reduced-motion`.

5.2 State management — `src/store/useFleetStore.js` (Zustand)

Action	Responsibility
<code>init()</code>	One-time bootstrap: loads reference data in parallel, sets <code>ready</code> , best-effort <code>hydrateLive()</code> . No mock fallback if the API isn't configured — surfaces an error state instead.
<code>refreshFromApi()</code>	Polls <code>/fleet</code> + <code>/ops</code> every 5s; preserves client-only route geometry across polls to avoid map flicker; guards against a backend poll flipping a locally-completed trip back to en-route.
<code>hydrateLive()</code>	For each EN_ROUTE emergency lacking geometry, computes OSRM route legs via <code>bestLeg()</code> (traffic-adjusted route selection, \$9); writes the result back via <code>api.writeRoute</code> .
<code>tick()</code>	Runs every 1s; advances each live vehicle by a congestion-scaled step; marks vehicle idle / emergency completed on arrival.
<code>createEmergency()</code>	Posts to <code>/emergencies</code> , refreshes fleet/ops, re-hydrates live geometry.
<code>reassignEmergency()</code>	Manual override; drops stale geometry so it re-hydrates against the new assignment.

5.3 Auth UI — `src/auth.js`

SSO tokens arrive on the URL from the parent Jamshedpur platform (`?sso_token=&sso_id_token=`), captured into `sessionStorage` and stripped from the URL. Role is derived from the `cognito:groups` claim via a client-side decode. **The code explicitly documents that this decode is for routing only** (`auth.js:12-13`): "the backend must verify the JWT signature against the pool JWKS before trusting it for authorization" — i.e., a forged/unsigned JWT can pick a UI route locally, but cannot authorize any real backend action, because the backend independently re-verifies every token (\$14.1). `devLogin()` is compiled in only for local `vite dev` runs (gated on `import.meta.env.DEV`); there is no in-app login form in production.

6. Backend — API endpoint inventory

All routes handled in `lambda/TransportApi/Function.cs`, entry point `FunctionHandler` (line 72). Unmatched routes return 404; uncaught exceptions return 500 and are logged with an `ERROR` -prefixed line picked up by the CloudWatch filter (\$14, \$17).

Method	Path	Table(s)	Auth
GET	<code>/health</code>	—	none
GET	<code>/reference/*</code> (locations, zones, hospitals, firestations, policy)	ReferenceData	none
GET	<code>/track/{id}</code>	TransportRequests, Fleet	track-token query param
GET	<code>/fleet</code>	Fleet	authed if auth is on
POST	<code>/fleet/{id}/status</code>	Fleet	admin
GET/POST	<code>/fleet/vehicles</code> , <code>/fleet/refuel-requests</code> , <code>/fleet/refuel</code>	Fleet, ReferenceData	FUEL key or admin
GET/POST	<code>/allotments</code>	Fleet, jamshedpur-users, ReferenceData	authed (GET) / admin (POST)
GET/POST	<code>/fuel</code>	Fleet	admin
POST	<code>/emergencies</code> — create, runs dispatch	TransportRequests, Fleet, ReferenceData	scoped API key or portal JWT
POST	<code>/emergencies/{id}/route</code>	TransportRequests	same as create path
POST	<code>/emergencies/{id}/reassign</code>	TransportRequests, Fleet	admin
GET	<code>/emergencies/{id}</code> , <code>/requests/{id}</code>	TransportRequests	none extra
POST	<code>/requests</code>	TransportRequests	scoped API key (EDUCATION/DELIVERY/ADMIN/CONSOLE)
POST	<code>/requests/{id}/review\ assign\ dispatch</code>	TransportRequests, Fleet	admin
POST	<code>/requests/{id}/complete\ cancel</code>	TransportRequests, Fleet	admin OR the request's own owner
GET	<code>/ops</code> — aggregate dispatch board	TransportRequests	authed; non-admin sees only own records
GET/POST	<code>/bookings</code> — shuttle booking	ShuttleCards, TransportRequests, ReferenceData	HR key or CONSOLE
GET	<code>/employees</code> , <code>/employees/{id}</code>	jamshedpur-users, ReferenceData	admin
GET	<code>/cards</code>	ShuttleCards	admin
POST	<code>/policy</code> — upload PDF, invokes policy-sync	S3, PolicySync Lambda	admin
GET	<code>/infra/metrics</code>	CloudWatch/X-Ray/Bedrock (via CloudWatchService)	admin or MCP key
GET	<code>/analytics/coverage-gaps</code> , <code>/analytics/insights</code>	TransportRequestsHistorySynthetic (isolated)	admin or MCP key
GET	<code>/powerbi/embed-token</code>	— (Azure AD + Power BI REST API)	open beyond global GET gate

Global auth gates (before per-route checks): non-admin API-key callers are restricted to their scope's allowed POST resources (`Function.cs:108-109`, via `Auth.CanPost`); a non-admin JWT (portal) user may only `POST /emergencies` and `POST /requests/{id}/cancel` (line 112). GET routes for employees/allotments/fuel/cards/powerbi require admin explicitly; fleet/ops require any authentication when auth is enabled.

7. Backend — dispatch & assignment algorithm

The authoritative dispatch logic runs server-side in `BuildEmergency` (Function.cs:1423-1586), invoked from `POST /emergencies` and from a periodic `TryDispatchQueued` sweep. A client-side mirror in `src/services/dispatchService.js` exists only for map pre-ranking/overlays and is not the record of truth.

7.1 Nearest-vehicle selection

`FindNearestVehicle` (Function.cs:1250-1264) iterates zones by proximity to the pickup point (Haversine sort) and, per zone, runs a DynamoDB `Query` against the Fleet table's `GSI1-zoneveh` index (`GSI1PK = ZONE#{zoneId}#VEH` , `begins_with(GSI1SK, "idle#{type}#"`) , `limit: 1`) — **first idle unit in the nearest zone wins**, not a global nearest-vehicle ranking across all zones. Falls through to the next-nearest zone if none found; returns `QUEUED` status if no zone has a match.

7.2 Kind-specific routing

Kind	Logic
Fire	Finds a <code>firetruck</code> ; fire station picked by zone match or nearest Haversine; single-leg route origin→scene.
Blood	Finds an <code>ambulance</code> ; blood bank resolved by explicit ID or nearest <code>bloodbank</code> -type location; 3-leg round trip (zone → requesting hospital → blood bank → back to hospital).
Medical (default)	Finds an <code>ambulance</code> ; hospital candidates filtered by <code>case_type</code> membership in the hospital's <code>specialties</code> list. Critical severity sorts by capability first, distance only as tiebreaker ; Normal/Urgent sort by distance alone. Returns <code>NO_HOSPITAL</code> if no specialty match exists.

7.3 Mass-casualty / multi-unit

Unit count is derived from a `mass_patient_threshold` policy value (default 3): above it, `units = ceil(patients / patients_per_ambulance)` , capped by a `max_units` policy value (default 10). All units share one `incident_id` but are dispatched independently — there is no cross-unit coordination beyond the shared ID.

7.4 ETA/route computation

`ComputeRouteEta` (Function.cs:1407-1421) calls the real OSRM public router server-side; on failure it falls back to Haversine distance ÷ a configurable `speed_kmh` policy value (default 28 km/h), both scaled by a server-side traffic multiplier.

8. Backend — DynamoDB schema

No non-PDF schema source exists in the repo (`docs/DB-Schema.pdf` is binary); the schema below is inferred directly from every `Ddb.Query / Scan / UpdateItem` call site in `Function.cs`.

Table	Key pattern & notes
TransportRequests	Single-table design for REQ/EMG/BK entities: <code>PK = {ENTITY}#{id}</code> , <code>SK = META</code> (audit rows use <code>SK = EVT#{timestamp}</code>). Five GSIs defined (status-queue, zone, source, vehicle) but the Lambda predominantly uses full-table <code>Scan</code> with filter expressions rather than querying them — the GSIs appear maintained but under-utilized.
Fleet	Vehicles (<code>PK=VEH#{id}</code>), drivers (<code>PK=DRV#{id}</code>), allotments, fuel logs — all <code>SK=META</code> or dated sub-keys. <code>GSI1-zoneveh</code> (<code>ZONE#{zoneId}#VEH / {status}#{type}#{id}</code>) is the one GSI actually queried, by <code>FindNearestVehicle</code> . Two further GSIs (driver status, vehicle status) are maintained on write but never queried in this file.
ReferenceData	PK-partitioned: <code>LOC</code> , <code>ZONE</code> , <code>HOSP</code> , <code>FIRE</code> , <code>POLICY</code> (multiple versions, queried sorted-descending, limit 1, for "current policy").
ShuttleCards	Card (<code>PK=CARD#{id}</code>) + ride sub-items; monthly cap enforced via a conditional <code>UpdateItem</code> .
jamshedpur-users	Employee directory — queried via full <code>Scan</code> with a filter expression rather than a direct key lookup.
TransportRequestsHistorySynthetic	Read-only, <code>Scan</code> -only, seeded offline. Fully isolated from live dispatch data by design (explicit code comment) — used only by the two analytics endpoints. See §14.3 for the IAM enforcement of this boundary.

9. Live routing & traffic simulation

Both the backend dispatch ETA and the frontend live-map animation use the same public OSRM demo server (router.project-osrm.org), independently:

- **Backend** (`lambda/TransportApi/OsrmService.cs`): 4-second timeout, per-invocation in-memory cache (dedup for mass-casualty multi-unit requests), returns `null` on any failure so the caller falls back to Haversine.
- **Frontend** (`src/services/osrm.js`): requests full route geometry for map rendering; on failure, a straight-line fallback with interpolated points keeps the animation smooth.
- **Simulated traffic** (`src/services/traffic.js`): a per-zone congestion factor (1.0–3.0) that follows a rush-hour Gaussian curve (peaks ~09:00 and ~18:00) plus a bounded random walk, refreshed every 4s. This is a client-side simulation, not a real traffic data feed.
- **Traffic-aware route selection** (`bestLeg` , `useFleetStore.js:11-21`): requests 3 OSRM alternative routes and picks whichever has the lowest simulated-traffic-adjusted duration. **Known limitation, flagged during this engagement:** there is currently no cap on how much longer a "winning" alternative is allowed to be in absolute distance — because the traffic factor is a noisy random walk rather than real data, an alternative route can occasionally win purely by chance even when it is a poor detour in absolute terms. A fix (reject alternatives beyond ~1.4x the shortest option's distance regardless of simulated traffic score) has been scoped but not yet applied — see §17.

10. AI features: VoiceAgent & Open Knowledge Format

`lambda/VoiceAgent/Function.cs` is the only component in the system that performs live LLM inference at request time. Only Amazon Bedrock is used — no Transcribe, Polly, or Lex; speech capture/playback (where used) is handled client-side by the browser's Web Speech API.

Step	What happens
1	CORS preflight; Bearer JWT verified against Cognito JWKS if enabled (conditional — see §16).
2	Conversation capped to the last 20 turns, each truncated to 1,000 characters, before anything reaches the model.
3	First turn → a scripted greeting; Bedrock is not called yet.
4	<code>ExtractSlots()</code> sends the transcript + system prompt (including the OKF bundle below) to Bedrock <code>Converse</code> ; the JSON reply is parsed defensively.
5	Extracted fields are validated against fixed, deterministic enums (<code>Critical/Urgent/Normal</code> ; <code>Cardiac/Trauma/General/Maternity/Pediatric</code>) — not the model's judgment .
6	Free-text location resolved to a known ID by fuzzy token-overlap scoring — plain string matching, not an LLM call.
7	A confirmation summary is shown; dispatch only fires once <code>confirmed: true</code> is received.
8	<code>DoDispatch()</code> posts the structured booking to TransportApi's <code>/emergencies</code> using a static <code>x-api-key</code> .

The load-bearing safety property: the LLM is used purely as a natural-language slot extractor. It never writes directly to the dispatch system, never picks a hospital, never sets severity outside the fixed enum. A hallucinated value is simply rejected and re-prompted, not passed through.

10.1 Open Knowledge Format (OKF) — the voice agent's data input

`LoadOkf()` (`Function.cs:57-79`) reads a small bundle of Markdown files from `infra/knowledge/` at cold start — `emergency-types/index.md` , `vehicles/index.md` , and every file under `locations/` — and concatenates them verbatim into the Bedrock system prompt on **every** call. If the directory is missing, it silently falls back to a plain `id=name` list from the live reference API.

What this is: static context injection ("prompt-stuffing") of a curated, version-controlled document set — simple, cheap, fully auditable (you can read exactly what the model saw by reading the Markdown files).

What this is not: there is no chunking, no embeddings, no vector similarity search, and no dynamic selection of which documents to include. Every call gets the entire bundle unconditionally. This works at the current small corpus size; it would not scale without becoming genuine retrieval (see §12.3).

11. AI features: MCP integration ("jsd-cloudwatch")

A local, on-demand developer tool — not a persistent network service, and not exposed to end users of the dispatch application.

Element	Detail
Server	infra/mcp/cloudwatch-server.mjs (226 lines), Node.js, registered in the git-ignored .mcp.json
Transport	stdio JSON-RPC 2.0 — spawned as a child process per request; no open port, no daemon
Backend proxied	Single endpoint: GET /infra/metrics on TransportApi
Tools	get_lambda_metrics , get_recent_errors , query_logs , get_infra_summary (thresholded healthy/degraded status), get_infra_metrics (raw payload)
Auth	Static x-api-key , mapped server-side to source label MCP , scoped to the infra resource only — cannot touch dispatch, fleet, or emergency data even if leaked, because the backend enforces the scope regardless of what the caller requests

Proposed but not built: the design doc docs/MCP-Security-And-Chatbot-Options.txt describes a second option — an in-app admin chatbot at POST /infra/chat using Bedrock. No such route, component, or API method exists anywhere in the codebase.

12. AI features: the "policy engine" & RAG status

Two unrelated features share the word "policy" — conflating them is the easiest mistake to make describing this system.

12.1 `infra/policy-agent` — Bedrock document extraction, wired end-to-end

Element	Detail
Trigger	<code>POST /policy</code> — admin-only, 5MB cap, PDF uploaded to S3, synchronously invokes <code>psiog-policy-sync</code>
Model call	Bedrock <code>Converse</code> with the raw PDF as a document content block — <i>one shot, no retrieval, no chunking</i>
Extraction target	Exactly 6 named parameters: <code>speed_kmh</code> , <code>mass_patient_threshold</code> , <code>patients_per_ambulance</code> , <code>max_units</code> , <code>autocomplete_minutes</code> , <code>severity_order</code>
Guardrail	<code>sanitize()</code> clamps every value to hardcoded bounds (e.g. <code>speed_kmh</code> ∈ [5,120]) before anything is applied
Apply step	Writes the sanitized JSON to TransportApi's <code>POLICY_CONFIG</code> Lambda environment variable
Consumption	Read at TransportApi cold start; feeds ETA calc (<code>speed_kmh</code>) and mass-casualty sizing (\$7.3) directly

The code's own comments are explicit: it controls parameters and does not generate or replace code. This is a narrow, bounded-output-space use of an LLM — not a general reasoning agent, and not RAG (there is no corpus; the entire input is one PDF, sent whole, every time).

12.2 `PolicyControls.jsx` / `policyService.js` — an unrelated feature, same word only

Manages fuel allowances and permitted vehicle grades per job level. Plain client-side config CRUD, zero AI involvement, no relationship to §12.1 beyond the shared English word.

12.3 On RAG specifically

There is no retrieval-augmented generation anywhere in this codebase. A repository-wide search for vector-store, embeddings, or chunking patterns returned no matches. If a genuine RAG capability were added later — e.g. to let the voice agent or an admin chatbot answer from a larger, frequently-updated corpus than the current small OKF bundle — it would need, at minimum: a chunking strategy, an embeddings model call (Bedrock Titan Embeddings), a vector index (OpenSearch Serverless with k-NN is the natural AWS-native choice), and a query-time similarity-search step before generation. None of that exists today.

13. AI features: statistical analytics (not ML)

Two GET endpoints, both reading only from the isolated synthetic table, both explicitly documented in code as "plain, explainable techniques" rather than a black box — no model inference of any kind.

Endpoint	Technique
/analytics/coverage-gaps	Per-zone average/P90 ETA, SLA-breach % against fixed severity targets, a fixed 1.4x gap-ratio threshold with a 30-call minimum sample size to flag a zone.
/analytics/insights	Placement: demand-weighted centroid of actual historical pickups per zone vs. the current staging point (haversine distance) — flags meaningful drift.
	Staffing: Little's Law ($L = \lambda \cdot W$) applied at each zone's historical peak hour, sized to 70% target utilization.
	Seasonal alerts: hardcoded real calendar-event tags (Omicron-wave window, Diwali, New Year's Eve) plus a dynamically computed monsoon road-trauma share ratio.

Both endpoints are gated to `admin || apiKeySource == "MCP"` only, and both responses carry a machine-readable disclaimer identifying the data as synthetic/demo, never live dispatch records. The underlying seed data (5,500 records, 5-year window) was deliberately built with real-world-grounded outliers rather than uniform noise — an industrial-zone fire/trauma skew, a real Omicron-wave date window, real Diwali calendar dates, and a monsoon trauma-share increase — so the recommendations above have an actual signal to detect rather than fitting noise.

14. Security architecture

14.1 JWT verification (Cognito JWKS)

Implemented identically in both Lambdas (`Auth.cs:68-100`, `VoiceAgent/Function.cs:377-405`): fetches and caches (1-hour TTL, double-checked-lock) the Cognito pool's JWKS, validates issuer, signature, token lifetime (5s clock-skew tolerance), optional audience/ `client_id` match, and restricts `token_use` to `access / id`.

14.2 API key scoping

`Auth.cs:31-52`: each key maps to a source label (`CONSOLE`, `HOSPITAL`, `EDUCATION`, `DELIVERY`, `ADMIN`, `HR`, `FUEL`, `MCP`), and a `Scopes` dictionary restricts each label to specific POST resources — only `CONSOLE` is wildcard. The two analytics GET routes additionally require `admin || apiKeySource == "MCP"` ; ordinary hospital/education/delivery/fuel keys cannot read analytics data at all.

14.3 DynamoDB synthetic/live data isolation

The live `TransportRequests` table and the synthetic `TransportRequestsHistorySynthetic` table are fully separate, never cross-referenced at request time. IAM reinforces the same boundary: `infra/sso-permission-set-policy.json` grants write access only to the synthetic table's ARN — the live table is read-only in that policy.

14.4 CORS, secrets, input validation

Control	Detail
CORS	<code>ALLOWED_ORIGINS</code> allowlist, defaults to <code>*</code> if unset — a deploy-time configuration responsibility, not a hardcoded restriction (see §16).
Secrets	<code>.mcp.json</code> is git-ignored (confirmed, never in git history). <code>API_KEYS</code> is currently a plaintext Lambda env var by default; a Secrets Manager migration exists in <code>infra/security/harden-aws.sh</code> but is a manual, opt-in CloudShell subcommand (see §16).
VoiceAgent input	20-turn / 1,000-char caps applied before anything reaches the model.
Policy PDF upload	5MB cap, valid-base64 requirement.
Extracted policy values	Hardcoded numeric clamps applied before any value reaches production config.
Forged-JWT risk	Client-side JWT decode is explicitly documented as routing-only (§5.3); the backend independently re-verifies every token against Cognito JWKS before authorizing any action.

15. Testing & QA

15.1 Frontend — Vitest (src/__tests__/)

File	Tests	Covers
sla.test.js	31	SLA target/status/text logic across policy overrides and queue states
locations.test.js	24	Location/zone/blood-bank lookups, proximity sort, label formatting
hospitals.test.js	12	Case types, severity metadata/ranks/colors, hospital lookup
dispatchService.test.js	15	Vehicle home position, nearest-zone-pool selection, driver assignment
backend-handler.test.js	47	Pure-function mirror of the C# handler's scope/validation logic (a JS reimplementaion used for fast unit tests, not the live Lambda code path): API-key scoping, emergency validation, Haversine correctness
Vitest total	129	

15.2 Frontend — Jest (src/__jest__/, jsdom environment)

File	Tests	Covers
LiveEta.test.jsx	9	Countdown rendering, clamping, arriving state, per-second tick
emergencyFilters.test.js	17	Active/completed/all filter logic, count accuracy
buildMetrics.test.js	16	Dashboard metric aggregation (totals, utilization, top hospitals)
VoiceAgent.test.jsx	5	Overlay render, error states, dispatch-confirmation gating
Jest total	47	

Known, expected split: Jest-config test files fail if run under Vitest ("describe is not defined") — this is pre-existing and by design; the two suites are invoked by separate npm scripts and run as separate CI steps.

15.3 Backend — xUnit (lambda/TransportApi.Tests/)

`DynamoServiceTests.cs` — 38 tests covering `DynamoService.cs`'s attribute-value conversions, including a specific regression test for an empty-collection guard bug (distinguishing the SDK's default-initialized empty `M / L` collections from real empty maps/lists). **Known local environment limitation** (confirmed live during this engagement): this machine has only the .NET 10 runtime installed, while the test project targets net8.0 — `dotnet test` builds successfully but fails at runtime-launch locally. CI installs .NET 8.0 explicitly and is expected to run these tests normally.

15.4 Combined totals & CI enforcement

`TEST_REPORT.md` (generated 2026-07-01) states a "Grand Total: 167" figure that in fact only sums Vitest (129) + xUnit (38) — the Jest total (47) was documented in a separate section added the same day and isn't included in that headline number. The true combined count across all three runners, as this report's own tables document, is **129 + 47 + 38 = 214 tests**. All three suites run automatically and gate deployment in `.github/workflows/deploy.yml`'s `test` job — both the frontend and Lambda deploy jobs declare `needs: test`. The older `buildspec.yml` (CodeBuild/CodePipeline) path has no test phase at all.

16. Deployment pipeline

Two parallel deploy paths exist. `.github/workflows/deploy.yml` (the more complete one) runs on push to `main`: a `test` job (Vitest, Jest, xUnit) gates two parallel jobs — `deploy-frontend` (Vite build → S3 sync → CloudFront invalidation) and `deploy-lambdas` (packages and deploys both TransportApi and VoiceAgent via `dotnet lambda package` + `aws lambda update-function-code`). Uses OIDC (no static AWS keys). `buildspec.yml` (CodeBuild/CodePipeline) covers only the frontend build/deploy, with **no test step**. Manual equivalents exist for CloudShell use: `infra/deploy-csharp.sh` (Lambda deploy) and `infra/setup-cli-user.sh` (one-time scoped IAM user bootstrap, used in place of AWS Identity Center/SSO which this account doesn't support in its current configuration).

17. Known gaps & technical debt

Auth is a runtime toggle, not a hard requirement

JWT enforcement in both Lambdas only activates if Cognito env vars are set at deploy time. If absent — a plausible misconfiguration, not a design intent — the endpoint falls through to API-key-only or fully open access. The signature verification itself is sound; the risk is entirely in deploy-time configuration discipline.

Traffic-aware route selection has no detour sanity cap

As detailed in §9: `bestLeg()` can occasionally select a genuinely long OSRM alternative purely because the simulated (random-walk) traffic factor favored it by chance, with no safeguard against an absolute-distance detour. A fix has been scoped (reject alternatives beyond ~1.4x the shortest option's distance) but not yet applied.

Default-open CORS, plaintext API keys, unautomated hardening

`ALLOWED_ORIGINS` defaults to `*` unless explicitly locked down. `API_KEYS` is a plaintext Lambda env var by default. `infra/security/harden-aws.sh` offers a Secrets Manager migration, CSP/HSTS headers, a WAF rate-based rule, voice-endpoint throttling, and a cost budget — every one is a manually-run CloudShell subcommand, not part of the automated pipeline. None of these should be assumed "in place" without independently confirming they were run against the live environment.

Legacy/dead-code candidate: `backend/`

`backend/handler.mjs` (1,052 lines) + `backend/auth.mjs` is a full Node.js reimplement of the same API surface as the current C# `TransportApi`, predating the C# rewrite commit. Not referenced by either deploy pipeline; still ships with its own deploy script. Strong candidate for removal pending confirmation it's unreferenced in the live AWS account.

DynamoDB GSIs defined but under-utilized

Five GSIs exist on `TransportRequests` (status-queue, zone, source, vehicle indexes) but most read paths use full-table `Scan` with filter expressions rather than querying them. Only `Fleet`'s `GSI1-zoneveh` is actually queried (by the nearest-vehicle dispatch logic). This is a cost/latency optimization opportunity as data volume grows, not a correctness issue today.

18. Summary matrix

Component	Status	Notes
Core dispatch (nearest-vehicle, hospital/blood-bank selection, mass-casualty)	LIVE	Deterministic, zone-proximity + capability-based; no AI involved
Live map, routing & traffic simulation	LIVE	Real OSRM road-network routing; traffic is a client-side simulation, not real data
VoiceAgent (Bedrock slot extraction)	LIVE	Every extracted field validated against closed enums / real reference data before use
OKF knowledge bundle	LIVE	Static Markdown prompt-stuffing, not retrieval — appropriate at current corpus size
MCP server (jsd-cloudwatch)	LIVE	Local stdio dev tool, scoped API key, read-only observability
MCP-based admin chatbot	PROPOSED ONLY	Documented design, no code exists
Policy-agent (Bedrock PDF extraction)	LIVE	End-to-end: PDF → 6 clamped parameters → live dispatch config
Retrieval-augmented generation	NOT IMPLEMENTED	No vector store, embeddings, or retrieval step anywhere in the repo
Coverage-gap / AI Insights analytics	LIVE, STATISTICAL	Little's Law, centroid geometry, percentiles — no model inference; synthetic data only
Auth (Cognito JWT + scoped API keys)	LIVE	Sound verification logic; enforcement is deploy-config-dependent (\$17)
Testing (214 tests across 3 runners, CI-gated)	LIVE	Local xUnit run blocked only by a missing .NET 8 runtime on this dev machine, not CI
Security hardening suite (harden-aws.sh)	AVAILABLE, NOT AUTOMATED	Manual CloudShell subcommands; unclear if exercised against the live account
Legacy Node.js backend (backend/)	LIKELY DEAD	Superseded by the C# rewrite; not in either deploy pipeline